

Cloud

Module 5 - Container



Input of this Module

- <https://learning.oreilly.com/playlists/43827098-7a87-435e-823e-736586b5694c>
05-container
- https://access.redhat.com/documentation/de-de/red_hat_enterprise_linux/8/html/managing_monitoring_and_updating_the_kernel/using-cgroups-v2-to-control-distribution-of-cpu-time-for-applications_managing-monitoring-and-updating-the-kernel
- <https://wizardzines.com/comics/>
- Several Blogposts

slido

Please download and install the Slido app on all computers you use



What is a container?

① Start presenting to display the poll results on this slide.

Containers are disruptive...

Performance:

- Minimal overhead of process

Easy scalability:

- Easy to scale horizontally without provisioning, startup, etc

Portability:

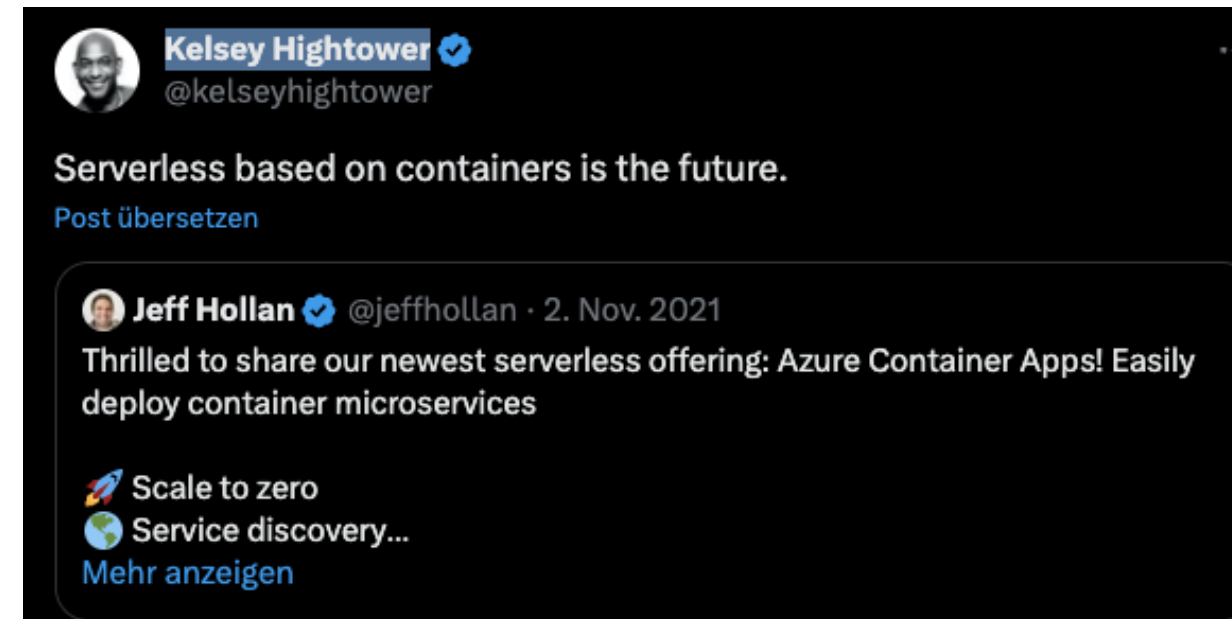
- Easy transfer of container to different hosts via http/s

Consistency:

- Self contained environment to run applications

Free and wide Ecosystem:

- Tools, platform is free and easy to use



...but what are containers?

JULIA EVANS
@b0rk

containers vs VMs



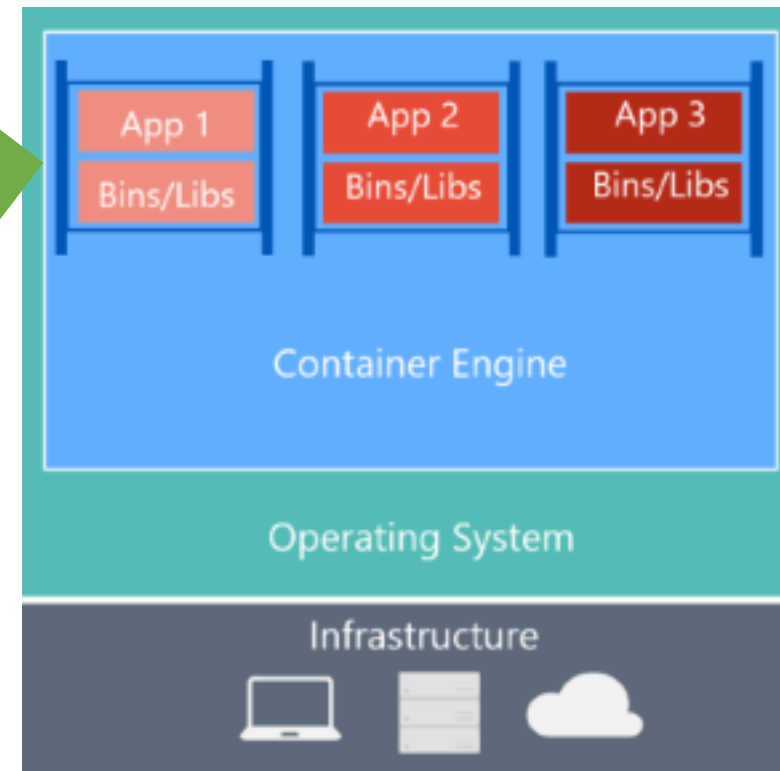
a container is a group of processes 1 Linux Kernel lots of containers computer	a virtual machine is a fake computer each one has its own operating system! Linux VM Windows VM BSD VM computer	containers use less RAM This is because they share a single Linux kernel. computer I can easily run thousands of small containers!
containers start faster because they're processes and process start fast ♥ Container done! VM um my operating system is still booting	containers are more complicated to secure VM I'm totally isolated from other VMs on this computer! container um it really depends how you configured me...	it's harder to figure out what you can do in a container VM just pretend I'm a computer! it's easy! container I act like a VM kinda but there are exceptions...

We all use Docker...

```

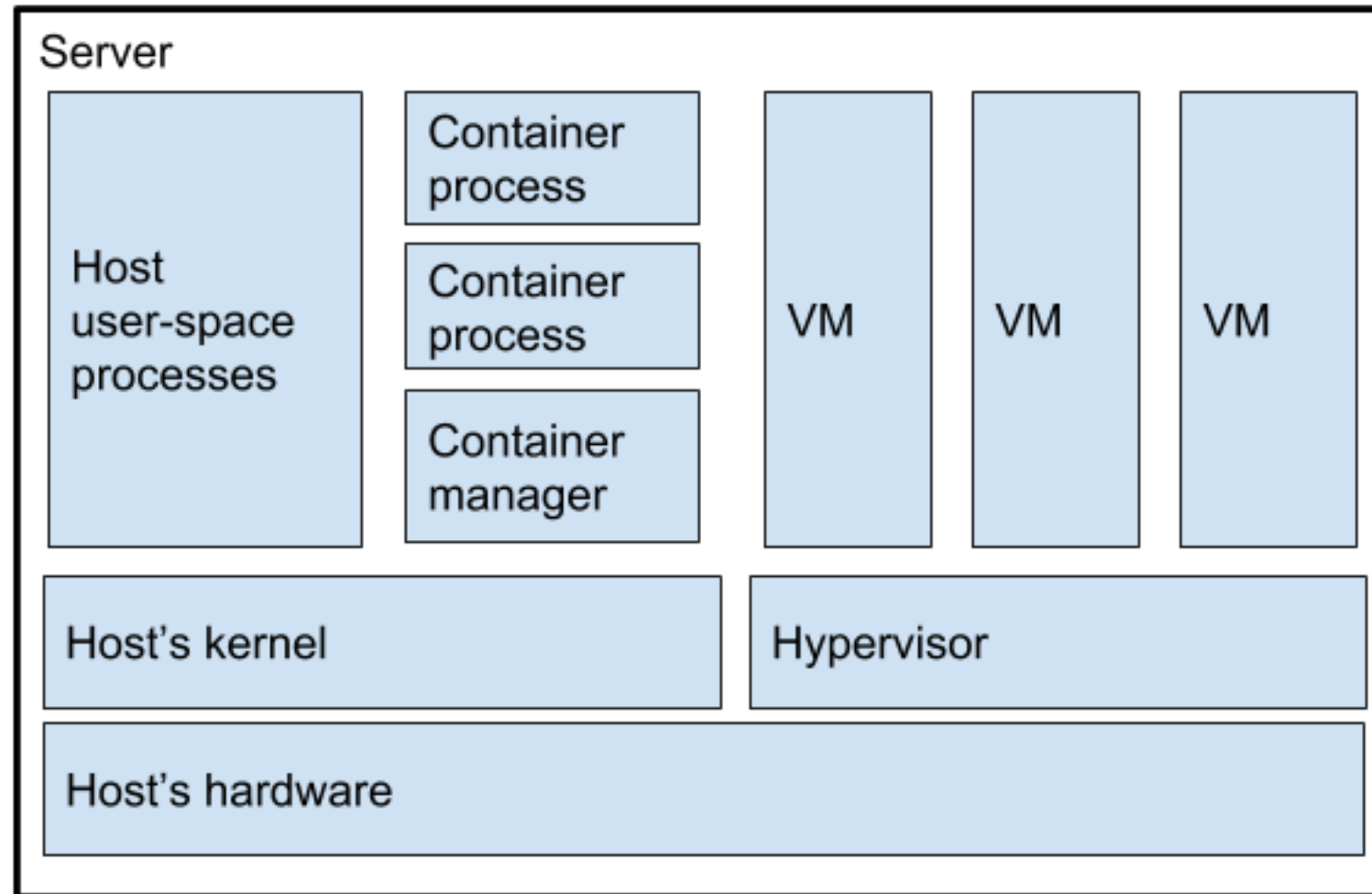
1 FROM ubuntu:15.04
2 COPY . /app
3 RUN make /app
4 CMD python /app/app.py
  
```

Line: 1:5 Dock... | Tab Size: 4



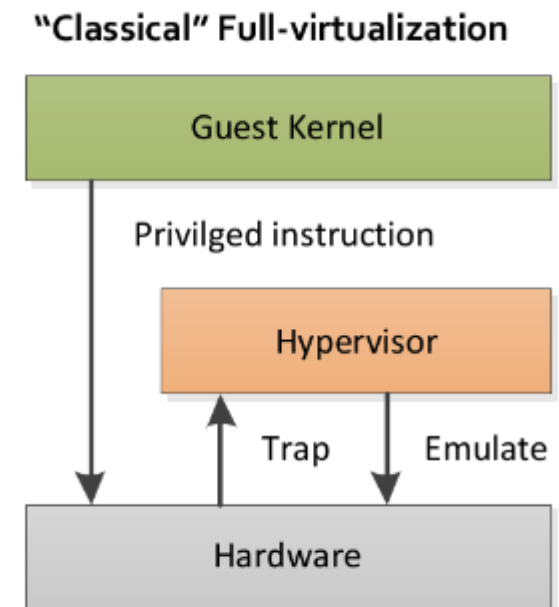
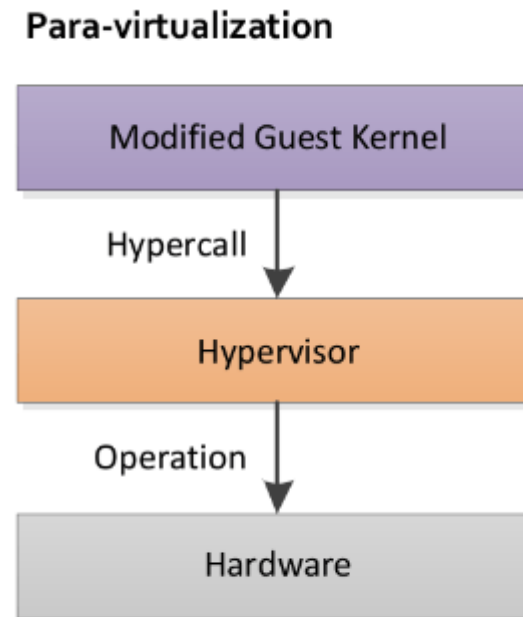
<https://docs.docker.com/storage/storagedriver/images/container-layers.jpg>

Containerization VS Virtual Machines



Recap: Paravirtualized Environments

- Guests entirely rely on hypervisor-enabled Kernel
 - In Practice: they share the same kernel
- Calls can be directly made to hypervisor (“hypercall”)
 - no interception / emulation necessary
- +: Less overhead, very efficient
- -: relies on kernel of host
- Famous products: lxc, xen, solaris zones

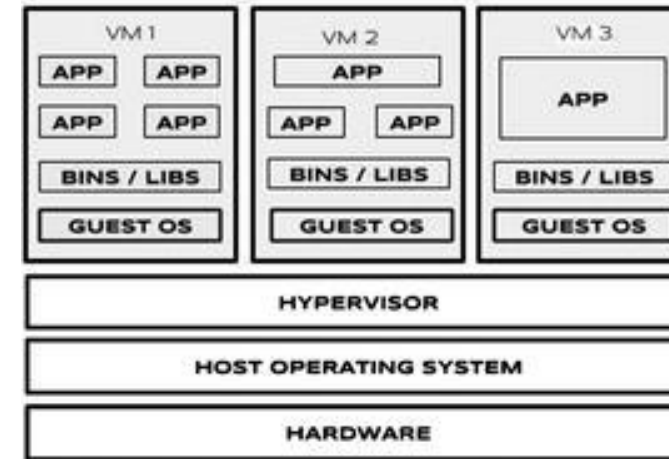


Special Kernel needed...but Is this really necessary for containers?

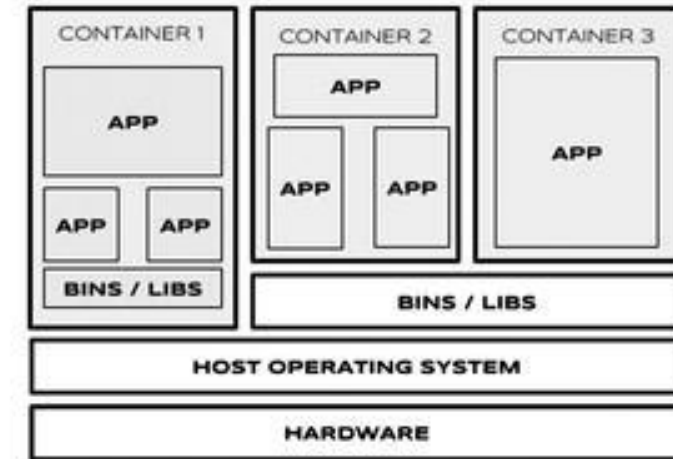
Containerization

Different Levels:

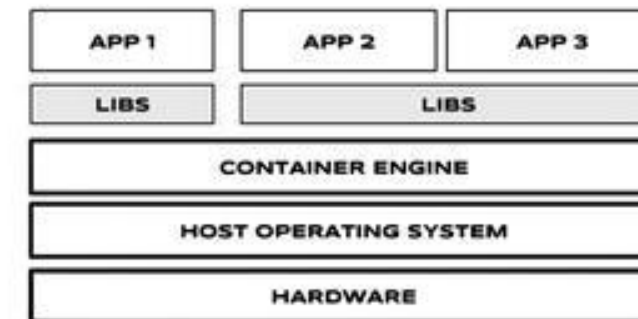
- Application Level:
Isolated Space where minimal processes run
- Operating System Level:
Entire operating systems runs in isolated space



Virtual Machines on Host



Linux Containers (lxc) or Operating System Level Containers



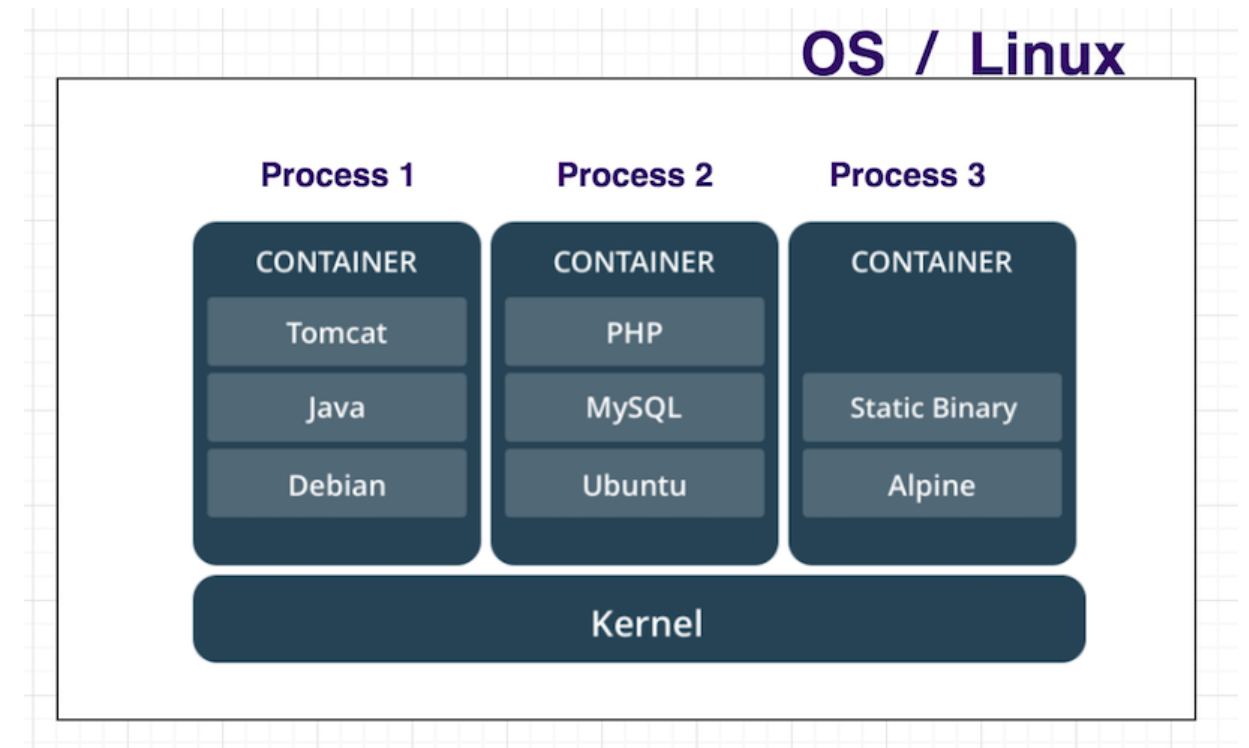
Application Level Containers

Under the hood: What is a container?

A container is a set of fixed defined processes.

Logical consequences are:

- Container have a lifecycle similar to a process.
- Containers have the same overhead than processes
- Containers have no additional start time (e.g. boot time)
- Containers leverage directly from kernel features for process handling / isolation
- Nevertheless, Containers are portable



Such containers are oooooooooooooold

Chroot:

- Still widely in use

LXC:

- relies on Kernel Modules starting 2.6.24
- Cgroups / chroot / namespaces

Docker:

- Started as a convenient interface for lxc

Year	Technology	First Introduced in OS
1982	Chroot	Unix-like operating systems
2000	Jail	FreeBSD
2000	Virtuozzo containers	Linux, Windows (Parallels Inc. version)
2001	Linux VServer	Linux, Windows
2004	Solaris containers (zones)	Sun Solaris, Open Solaris
2005	OpenVZ	Linux (open source version of Virtuozzo)
2008	LXC	Linux
2013	Docker	Linux, FreeBSD, Windows

How to implement a Container from scratch?

Filesystem should be isolated in the container:

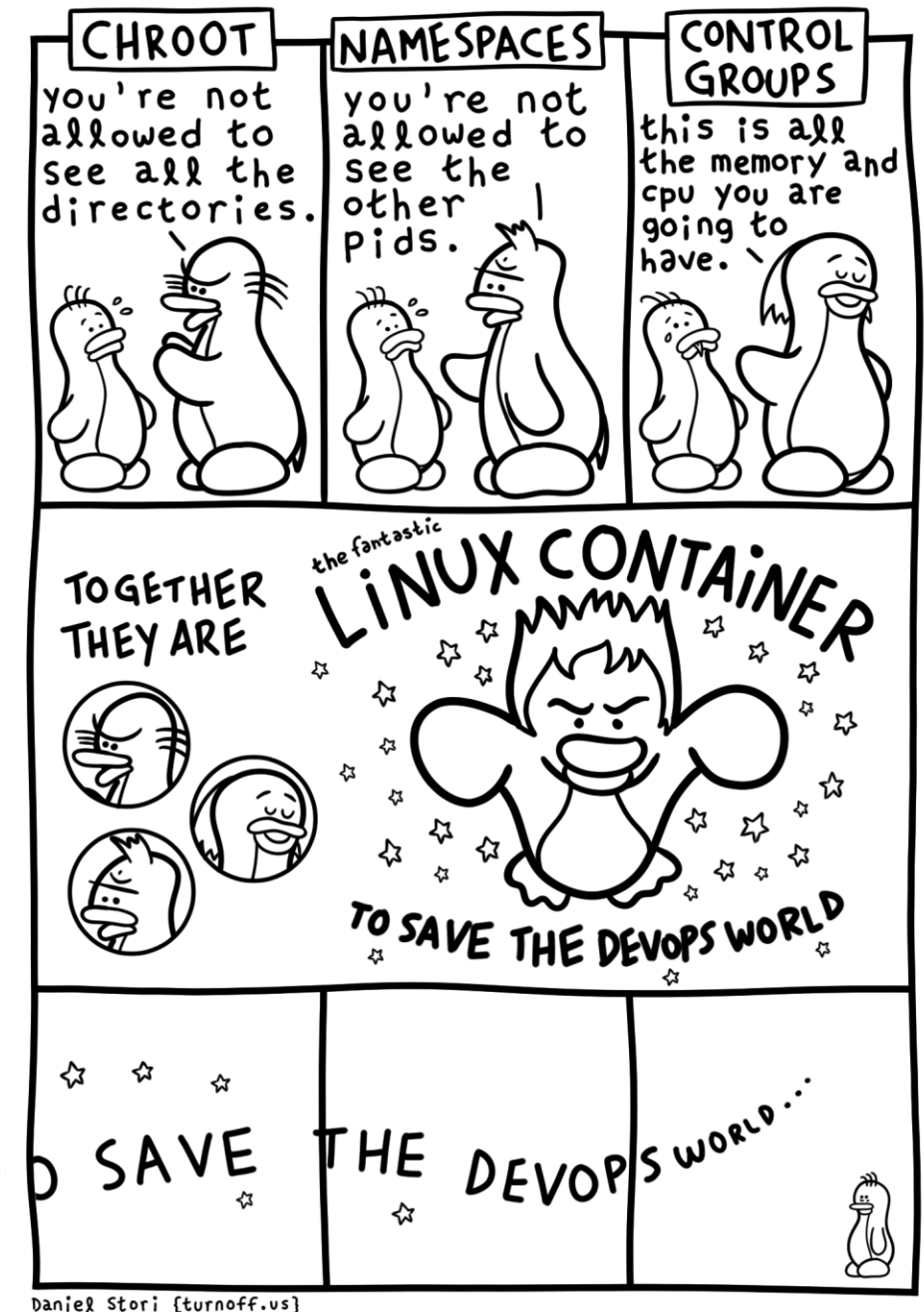
→ Chroot

Processes should look like they are only visible in the container:

→ Namespaces

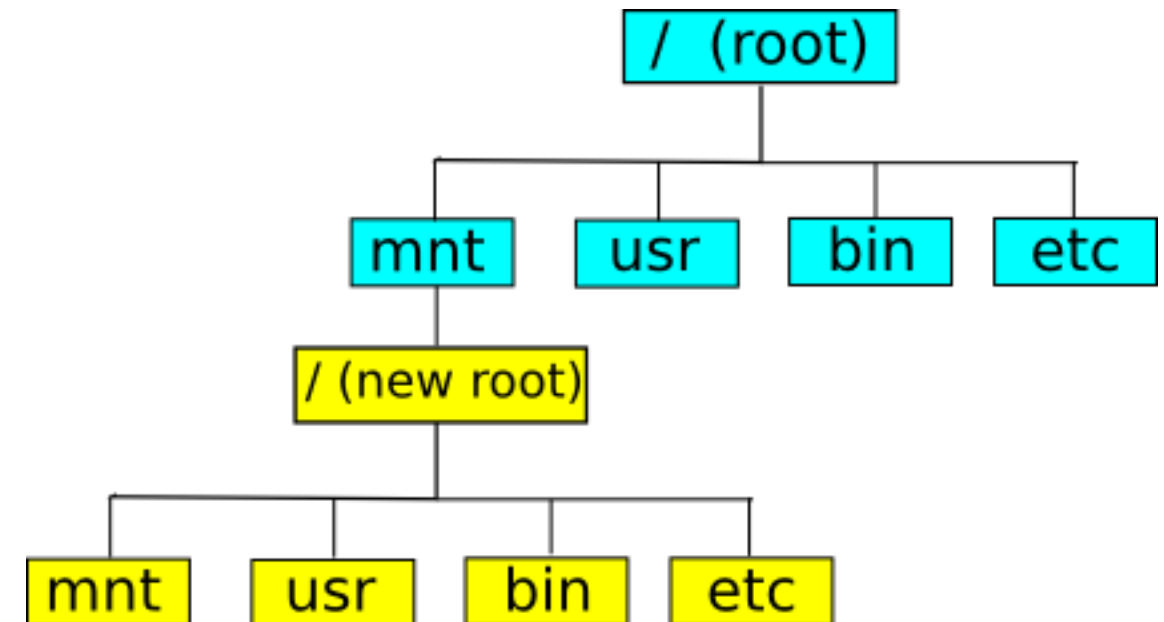
Resources should be restricted to the container:

→ cgroups



Chroot

- Chroot = change root
- Sets actually the root directory including all paths according to the File Hierarchy Standard to a non-root folder ($\neq /$)
- Combined with Mount Namespaces, process can act entirely isolated.



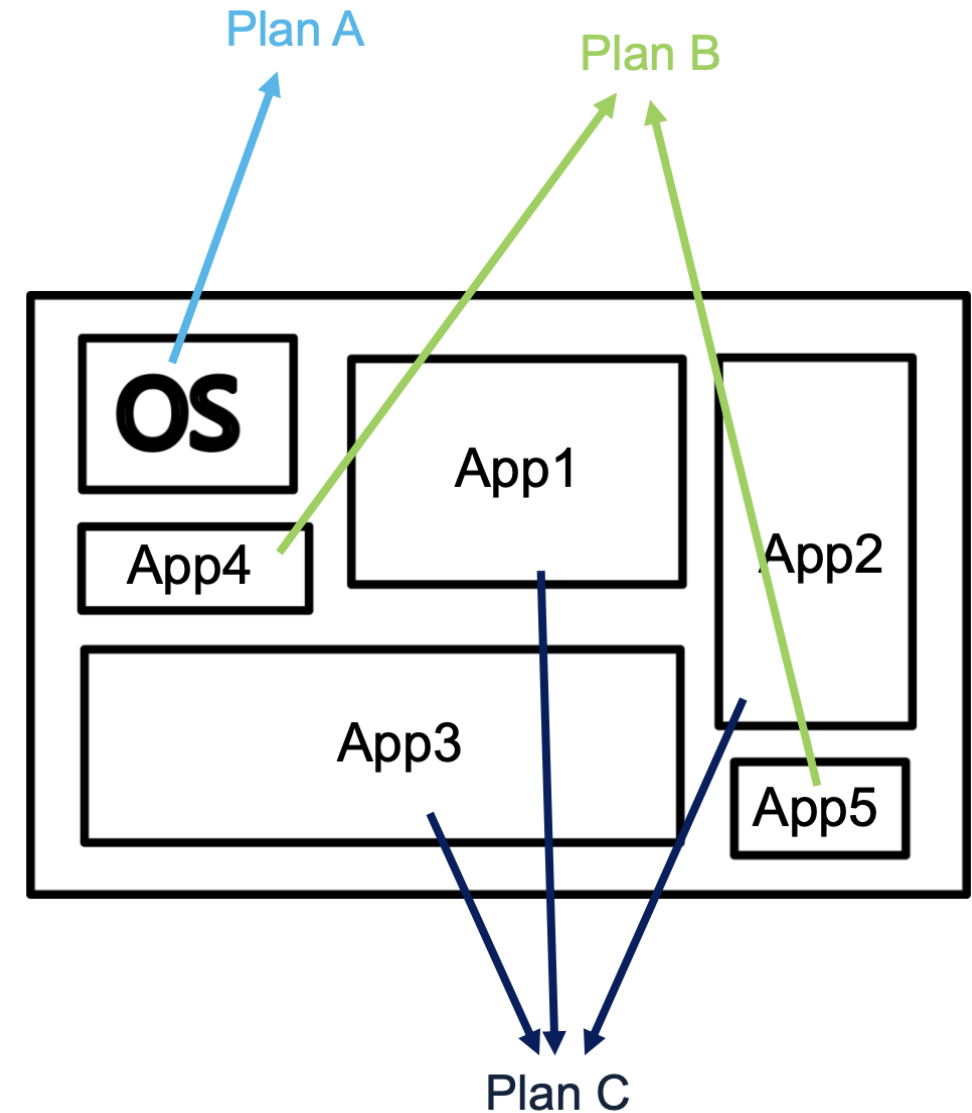
Cgroups

Different plans demand different resources.

App specs are according to cgroups:

- CPU time
- System memory
- Disk bandwidth
- Network bandwidth
- Monitoring

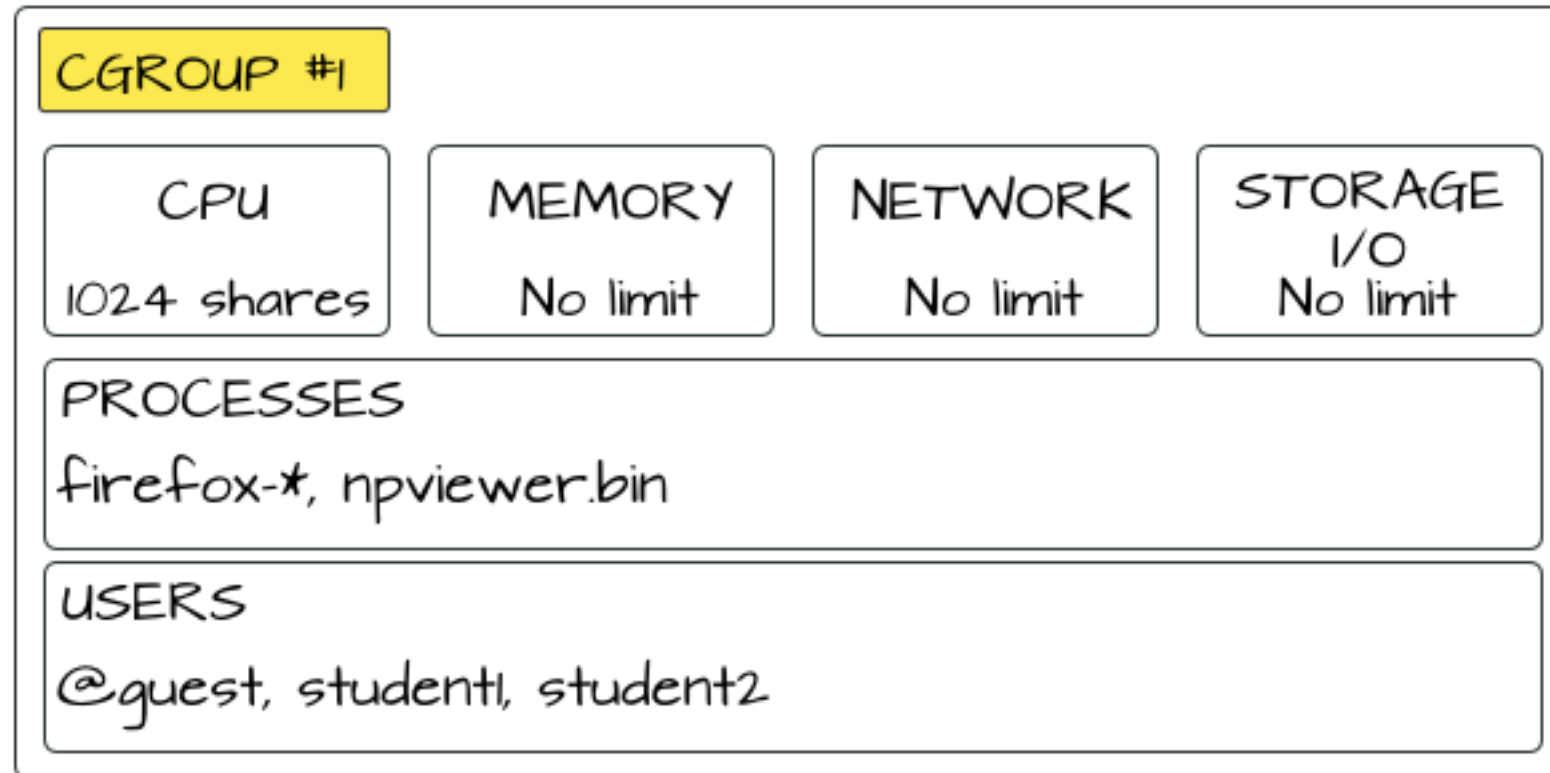
Cgroups (Control Groups) are organized in an hierarchy and summarize processes



What does a cgroup look like?

Different levels for different resources.
Refer to <https://docs.kernel.org/admin-guide/cgroup-v2.html> for all possibilities.

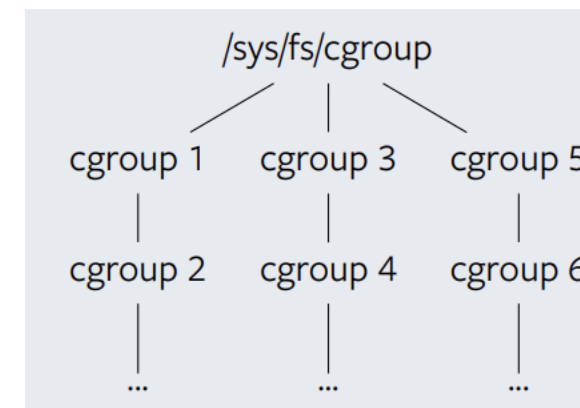
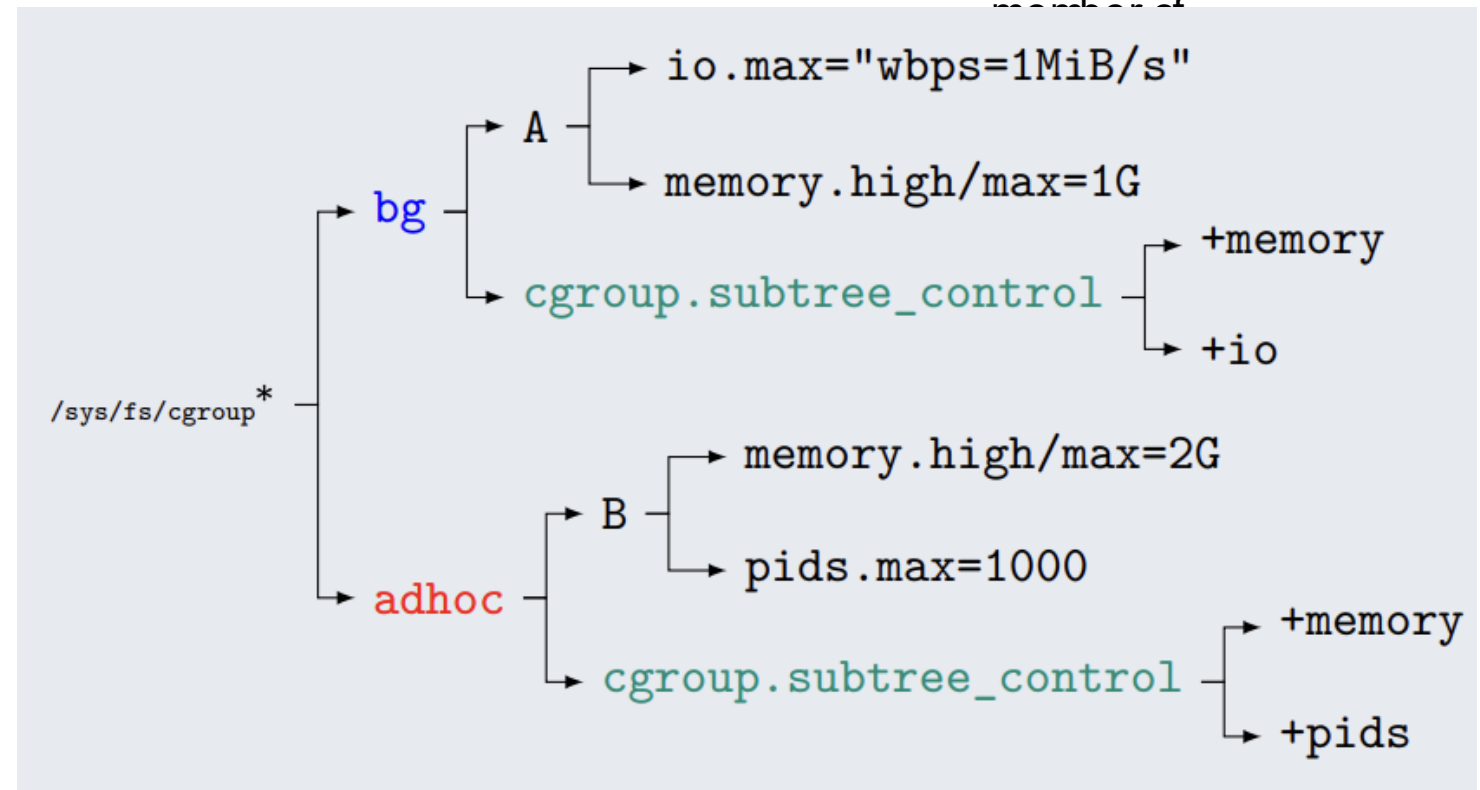
Beware of cgroup.v1 vs cgroup.v2



Hierarchy on file system

VFS under `/sys/fs/cgroup`

- Each cgroup contains rules as files
- Each cgroup can be enabled for dedicated rules
- A cgroup can contain other cgroups



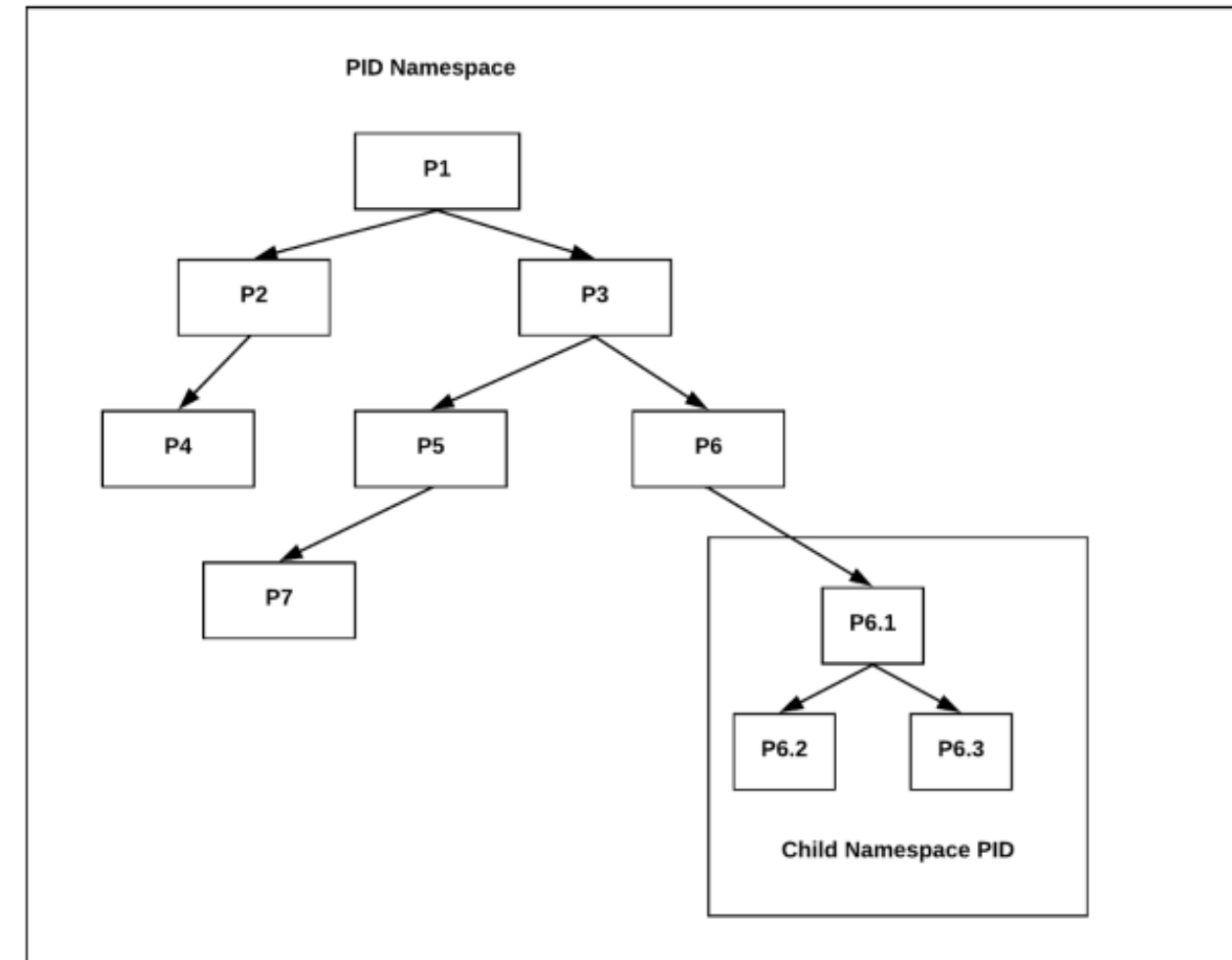
Namespaces

VFS under `/proc/PROCESS`

- Each process contains all necessary information about itself
- E.g. adaption of cgroup

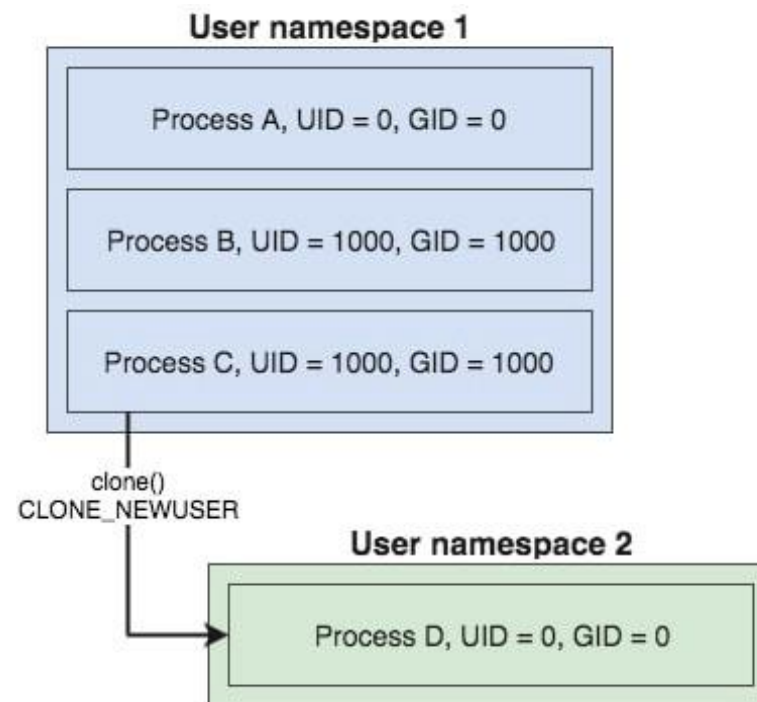
Different system calls to control:

- `clone()`: This creates a new process and attaches it to a new specified namespace
- `unshare()`: This attaches the current process to a new specified namespace
- `setns()`: This attaches a process to an already existing namespace



Namespaces

- Mapping of UIDs/GIDs between Child-Id and global id possible:
see `/proc/<PID>/uid_map` and `/proc/<PID>/gid_map`
- User in child process assume it is root



username	UID	GID
root	0	0
non-root-user	1000	1000

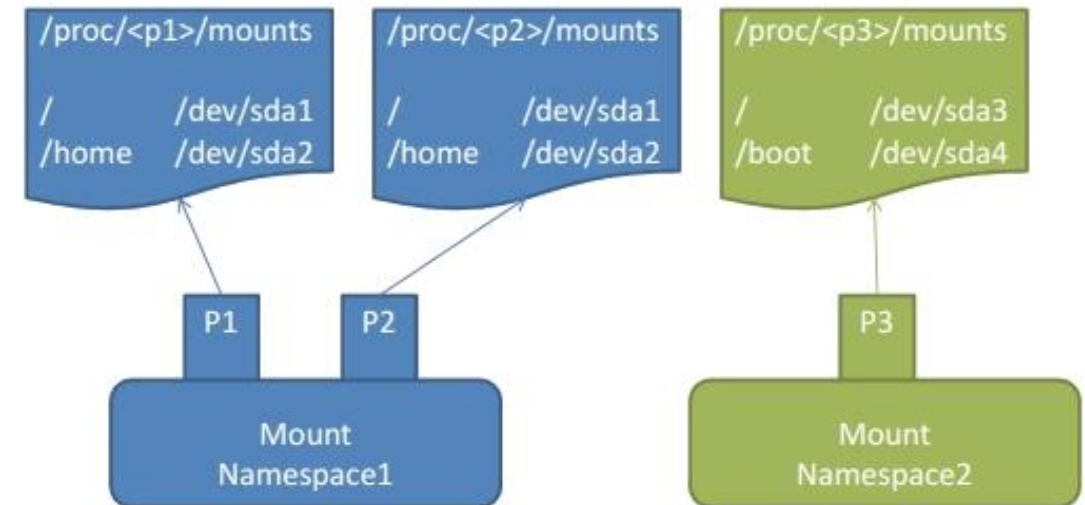
username	UID	GID
root	0	0

Many different namespaces

- **PID namespace:** isolation of the system process tree;
- **NET namespace:** isolation of the host network stack;
- **MNT namespace:** isolation of host filesystem mount points;
- **UTS namespace:** isolation of hostname;
- **IPC namespace:** isolation for interprocess communication utilities (shared segments, semaphores);
- **USER namespace:** isolation of system users IDs;
- **CGROUP namespace:** isolation of the virtual cgroup filesystem of the host.

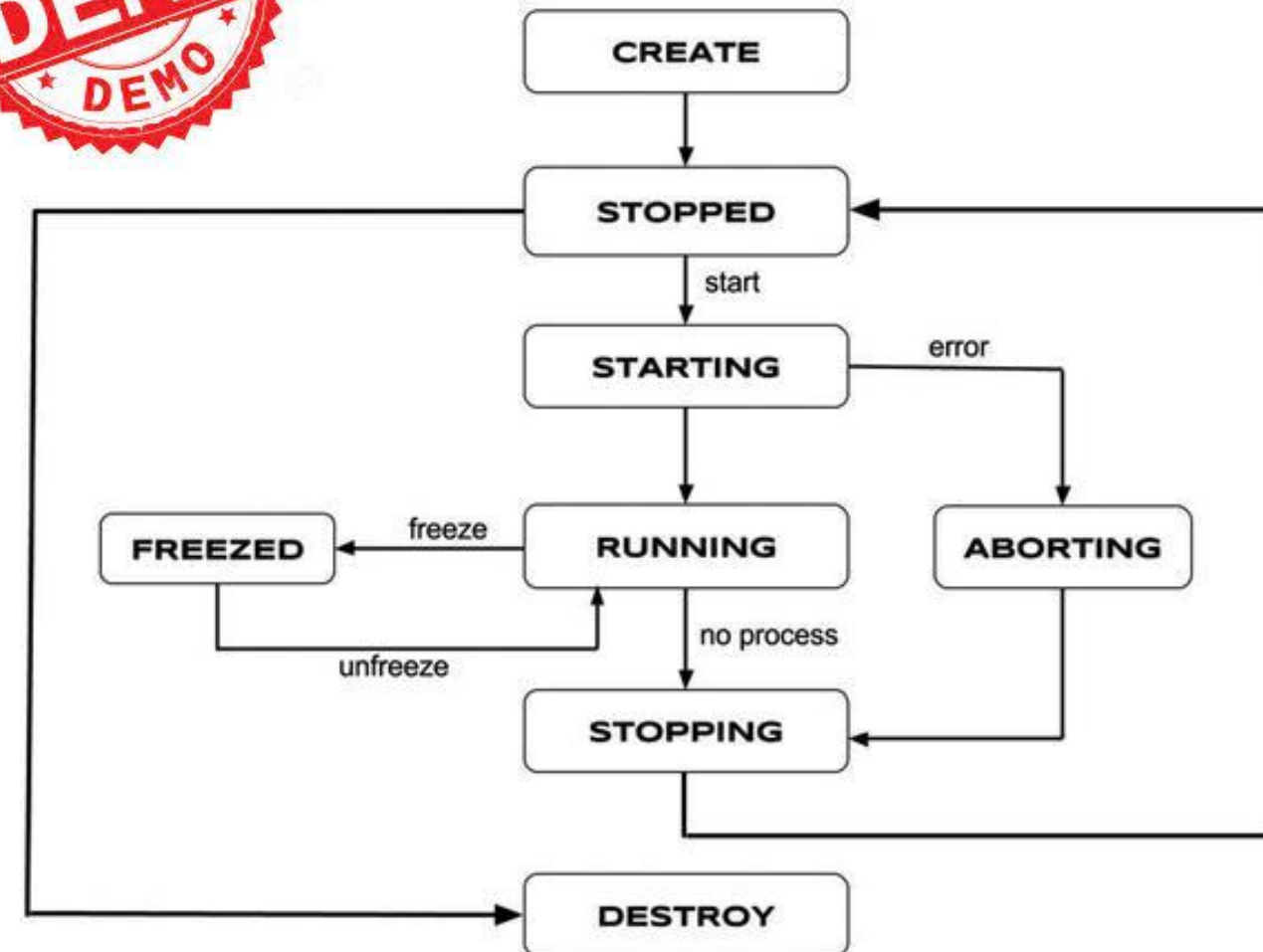
Mount Namespaces

- Namespaces can have dedicated mounts
 - Processes in these namespaces are isolated with respect to the mounted block devices
- Shared mounts between namespaces are possible though



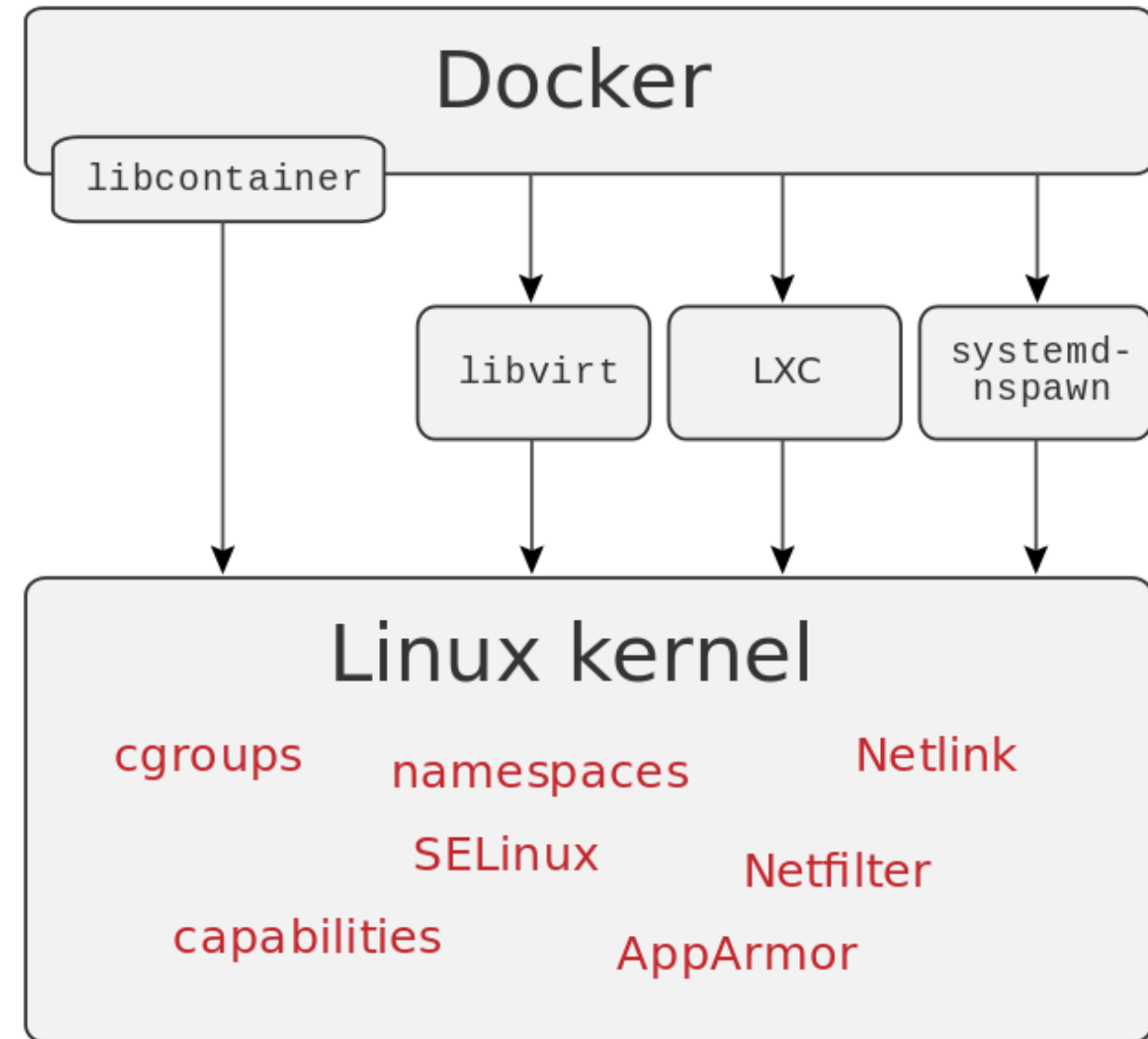
Putting it all together, lxc/lxd

1. `lxc-create`: Create a container with the given OS template and options.
2. `lxc-start`: Start running the container that was just created.
3. `lxc-ls`: List all the containers in the host system.
4. `lxc-attach`: Get a default shell session inside the container.
5. `lxc-stop`: Stop the running container, just like powering off a machine.
6. `lxc-destroy`: If the container will no longer be used, then destroy it.



Docker

Docker (as well as all containers) do not really virtualize / emulate:
They rely on kernel features only provided via different libs



Summarizing Questions

- What technologies / kernel features are actually needed for having containers?
- Do container emulate? If so, why is it necessary? If not, why is it not necessary?
- I want to restrict the memory usage of a container: what feature of the kernel enables me to do so?
- I want to install software in my container: why do mount namespaces protect my host?
- Why do PID-namespaces the usage of a single root-process in an application container?